BharatTrip take-home: prompt history

AI Operations Associate · every prompt I gave the AI coding assistant (Claude Code), in order, copied exactly as typed, with a one-line note on what came out of it. Screenshots I sent are described in [brackets]. Attached file paths are replaced by the file names.

How I worked with AI

- **Hypothesis first, then verify.** I gave my own first-pass diagnosis and asked the AI to prove or disprove each point with the data, and to stop for my review before building anything.
- **Clear division of labour.** Matching and money are plain code; AI only reads messy text and writes messages; a person approves every output.
- **I made the judgement calls.** Metric definition, whether to name handlers, the 7-day threshold, hosting and AI provider were my decisions, after the AI laid out the options.
- **Tested live and fed back real errors.** I deployed the app, used it as an operations user would, and sent back exact error messages and screenshots until it worked.
- **Designed for the end user.** I pushed back when the screen was built for a developer rather than an operations user.

Prompts (18)

PROMPT 1 · BRIEF AND PLAN

```
[Attached: BharatTrip_Refund_Data.xlsx and BharatTrip_Task_Brief.pdf] I'm doing a take-
home for an "AI Operations Associate" role at BharatTrip, a B2B travel company.
Attached: the task brief (PDF) and BharatTrip_Refund_Data.xlsx (tabs: Read_Me,
Support_Tracker, Finance_Tracker, Escalations). Read both fully before doing anything.

Context: Support logs refund requests in one sheet, Finance logs payouts in another.
They were never reconciled. Some requests come by email/WhatsApp and never get logged.
Escalations rose for two months while refund volume stayed flat (~150/month). Leadership
wants escalations down without adding headcount.

My first-pass hypothesis (verify or disprove, don't assume it's right):
- Support "Closed" doesn't mean paid. ~72 tickets are Closed in Support with no Finance
record, jumping from ~3-6/month to ~22-25/month in May-June, mostly from the Portal
channel.
- Escalation growth comes from new failure types (handoff breaks, never-logged informal
requests, approval delays, unexplained deductions), not from the old "no status update"
type.
- Most amount gaps are explained by Finance deductions that were never told to the
agent.
- Finance pending payouts all arrived in June; payout time is rising; Approved By is
mostly blank.

Phase 1: Clean and reconcile (Python/pandas, show me the code)
1. Normalise IDs (e.g. "RF 1750" -> "RF-1750"), parse mixed date formats (ISO, dd-mm-
yyyy, dd/mm/yy), trim and standardise status values, standardise agent names (case and
spelling), and remove exact duplicate rows. Log every cleaning rule and how many rows it
affected.
2. Join Support and Finance on the ID. Give each refund one "true state": Paid / Pending
in Finance / Declined / Closed in Support but missing in Finance / In Finance only /
Open in Support. Flag amount mismatches and whether the Deduction column explains them.
3. Sort every escalation into a root-cause category and show the categories by month.
Flag escalations whose ticket exists in neither tracker.
4. Find what changed in May: break the handoff failures down by channel, handler and
month. Show handoff time (Support request -> Finance received) and payout time (received
-> processed) by month.
Output: a short findings summary with numbers, plus a list of assumptions. Stop here and
wait for my review before Phase 2.
```

```
Phase 2: Prototype (a Streamlit app I can host and link)
- Upload the two trackers -> the same rule-based reconciliation -> a daily problem-case
list showing owner, age in days and the next action (e.g. "Closed in Support, not in
Finance: send to Finance").
- AI intake: paste an informal email/WhatsApp message (handle Hinglish) -> the Claude
API pulls out agent, route, amount, date and booking reference into JSON -> fuzzy-match
to existing records -> either link to a ticket or create a new draft ticket with an ID.
Test on the three informal messages in the brief.
- AI agent update: for any refund, draft a short status message to the travel agent from
its true state, explaining any deduction plainly.
- Design rule: matching and money calculations must be plain code, never the AI. The AI
only reads messy text and writes messages. Every AI output shows the source it was based
on, and a human approves it before anything is sent.
- Keep it simple. Clear README with a one-line run command. API key from an environment
variable.

Phase 3: Help me draft
- A 3-page write-up covering: problem diagnosis, analysis, solution design (single
source of truth = one refund register keyed on refund ID with a status shared by both
teams; what to automate and what to keep manual; how the teams stay in sync) and
metrics. Metrics: (1) count of refunds Closed in Support but not paid in Finance for
more than X days, target near zero, (2) escalations per 100 refunds, target back to
March-April levels within a month.
- Plain language. Every claim backed by a number from Phase 1.

I'm not a developer. Explain decisions briefly as you go, and ask before making any
judgement call that changes the conclusions.
```

Outcome: Read the brief and every tab of the workbook first. Built the cleaning and reconciliation code and the Phase 1 analysis. Results: 69 unique Closed-but-missing refunds (72 raw rows, 3 duplicates); failure rate jumped from 2-4% to 19% in May and 33% in June; escalations per 100 refunds went 15.1 to 52.8. Hypothesis checked point by point: 'mostly Portal' was misleading (every channel rose; two handlers' queues hit hardest), and blank Approved By was not new. Stopped for review with three questions.

PROMPT 2 · PHASE 1 REVIEW

```
1. add the 12 pending in finance, as technically its a wrong assumption
2. Fix is the process, but i guess we should keep the names in for visibility, suggest
your reasonings
3. Yes lets keep 7 days
```

Outcome: Metric 1 redefined to 81 refunds (69 missing + 12 still pending in Finance); 63 older than 7 days. Names kept, always shown with rates and the fact that every handler rose. Built the Streamlit prototype (worklist, AI intake, AI agent update) and drafted the 3-page write-up.

PROMPT 3 · HOSTING AND AI PROVIDER

```
lets connect vercel and deploy there , make the url live also API key change it to
gemini, for a free tier plan
```

Outcome: Explained that Vercel cannot host a Streamlit app and asked me to choose. My answers: host on Streamlit Community Cloud; use Gemini Flash. Switched the AI layer from Claude to Gemini; key read from GEMINI_API_KEY.

PROMPT 4 · LIVE TESTING

> `[Screenshot of the hosted app: 'AI call failed: Cannot send a request, as the client has been closed.']`

Outcome: Reproduced the error locally: a temporary Gemini client was being closed mid-call. Fixed and pushed.

PROMPT 5 · LIVE TESTING

> `AI call failed: Cannot send a request, as the client has been closed.`

Outcome: Made the fix robust (one client per call, kept open for the whole request), pinned the Gemini library and added a build label so I could confirm the live app had updated.

PROMPT 6 · LIVE TESTING

> `AI call failed: 404 NOT_FOUND. {'error': {'code': 404, 'message': 'models/gemini-flash-latest is not found for API version v1, or is not supported for generateContent. Call ModelService.ListModels to see the list of available models and their supported methods.', 'status': 'NOT_FOUND'}}`

Outcome: Replaced the guessed model name with code that asks Google which models the key can use and picks the newest stable Flash model.

PROMPT 7 · LIVE TESTING

> `AI call failed: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}`

Outcome: Added retries and automatic fallback to other Flash models, with tests that simulate a busy model.

PROMPT 8 · LIVE TESTING

> `[Screenshot: 'Drafting…' spinner on the agent update screen] it has been drafting and buffereing from past 2 mins`

Outcome: Found there was no time limit on requests. Added a 30-second limit per request and a 75-second overall cap, verified against a server that never replies.

PROMPT 9 · LIVE TESTING

> `AI call failed: Gemini is busy right now (no answer within 75s). Please try again in a minute. Last error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}`

Outcome: Changed strategy: a busy model is skipped at once instead of retried; up to six Flash and Flash-Lite models are tried; the last working model is remembered; the error lists every model tried.

PROMPT 10 · LIVE TESTING

> `[Screenshot: drafted message for RF-1042 with 'Number check passed'] msg draafted , but took around ~1 min`

Outcome: Checked the draft against the facts (accurate, honest, no invented numbers). Added a line showing which model answered and how long it took, and shortened the per-model limit to 20 seconds.

PROMPT 11 · CHECKING THE MATCHING

> `[Two screenshots: the Peak Journeys email and the 'Create DRAFT-001' decision with candidate refunds] the two windows compare each other, and the final payout amount should be 21300?`

Outcome: Explained why not: the email states no amount and the AI may not guess one. RF-1145 (Rs 21,300) is a possible match logged after the email, but Finance recorded a Rs 1,700 deduction, so its payout would be Rs 19,600. Added a comparison table with payouts worked out by code and a 'logged after the message' flag.

PROMPT 12 · DESIGNING FOR THE USER

> `[Screenshot: the raw JSON output on the intake screen] a non tech guy would use the dashboard, he doesnt need to know the JSON and all, he need the analysis, coparison and accurate amount with proof to be entered for the next stage`

Outcome: Redesigned the intake screen in three plain steps: what the message says (with quotes as proof), what our records show, and what goes to the next stage, with the amount and its source (Rs 21,300 - Rs 1,700 = Rs 19,600). JSON moved to a collapsed audit section.

PROMPT 13 · DESIGNING FOR THE USER

> `[Screenshot: 'Saved: linked to RF-1145' with no download button visible] no download button?`

Outcome: Found the cause (the sidebar was drawn before the approval was processed). Added a download button right under the confirmation and made the sidebar refresh.

PROMPT 14 · SPEED

> `the draft msg with AI takes too long, suggest ways to improve`

Outcome: Gave seven ranked options with effort and effect, and recommended the first four.

PROMPT 15 · SPEED

> `lets go with 1-4 for the time being`

Outcome: Implemented: minimal 'thinking', Flash-Lite first for drafting, message streamed as it is written, and saved drafts per refund. Each change covered by a test.

PROMPT 16 · POLISH

> `great, now lets polish the prototype, enhance the UI/UX and build it operations friendly simple to use and not confusing to a first time viewer`

Outcome: Reworked the app for first-time users: purpose and 'how to use' at the top, task-named tabs, plain-language worklist ranked by priority, a case view showing what Support and Finance each recorded, and step-by-step reply.

PROMPT 17 · ENTERPRISE READINESS

> `good, now go through the site, remove the emoji's, work as a senior UI/UX developer and designer,access the tool enhance it, act like a tester fix the bugs and then make it a production ready enterprise level tool`

Outcome: Tested every path and fixed eight bugs (including two crashes). Removed all emojis and applied one design system. Added upload validation, named approvals with an append-only log and its own tab, plain-language AI errors, a locked-down configuration, pinned libraries and automatic tests on every push. Listed the remaining gaps (single sign-on, database, live data feed) honestly in the README.

PROMPT 18 · SUBMISSION

> `make a pdf for the prompt history submission`

Outcome: This document.

Appendix: prompts the prototype sends to Gemini

These are the system prompts inside the app (ai.py). The app also forces Gemini's reply into a fixed JSON schema for message reading, and checks every amount in a drafted message against the facts in code.

1. Reading an email or WhatsApp message

```
You read informal refund messages that reach BharatTrip, a B2B travel company, by email or
WhatsApp.
Messages may be in English, Hindi written in Latin script, or a mix (Hinglish), e.g. "bhai
refund ka kya hua" = "brother, what happened to my refund", "2 hafte ho gaye" = "it has been 2
weeks", "koi jawab nahi" = "no reply".

Extract only what is written. If a field is not in the message, return null; never infer or
invent an amount, date or reference.
Do not convert relative times into dates; copy the phrase into time_reference. The software
does the date maths.
For every field you fill, add an evidence item quoting the exact words you used. If the sender
line (e.g. "from Peak Journeys") names the agency, that counts as written.
```

2. Drafting a status message to the agent (the facts are computed by code and sent with it)

```
You write short status updates from BharatTrip's refunds desk to a travel agent (a B2B
customer).
Use ONLY the facts provided. Do not add amounts, dates, reasons or promises that are not in
the facts.
Rules:
- 3 to 6 sentences, warm and plain. No jargon, no internal team names ("Finance tracker",
"true state").
- Always quote the refund reference.
- If there is a deduction, state the requested amount, the deduction and the amount paid, and
the reason if one is given. If no reason is given, say the team will confirm what it was for.
Do not invent one.
- If the refund is not paid yet, say honestly where it is and what happens next. Never say it
is paid when the facts say it is not.
- Write amounts exactly as given, in the form Rs 12,345.
- Sign off as "BharatTrip Refunds Team". Output only the message.
```